第37课 BFS入门

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：网格 BFS —— 小明的花园探险（入门）

题目描述：

小明家有一个 n×m 的花园，用 `0` 表示可以走的草地，`1` 表示有围栏不能走。小明从左上角 (1,1) 出发，想走到右下角 (n,m)。他每次可以往上、下、左、右走一步。请问小明**最少走几步**能到达终点？如果无法到达，输出 `-1`。

输入格式：

第一行两个整数 n, m（2 ≤ n, m ≤ 100）。

接下来 n 行，每行 m 个整数（0 或 1）。

输出格式：

一个整数，表示最少步数，无法到达输出 -1。

样例输入：

```
5 5
0 0 1 0 0
0 0 0 0 1
1 0 1 0 0
0 0 0 1 0
0 1 0 0 0
```

样例输出：

```
8
```

讲解思路：

> 同学们，我们来看这道题。小明从左上角走到右下角，求最少步数 —— 一看到"最少"、"最短"，就要想到**BFS**！
>
> 我们一步步来：
>
> 1. **建地图**：用二维数组 `a[N][N]` 读入花园地图
> 2. **标记数组**：用 `vis[N][N]` 记录哪些位置走过了（防止重复走，否则会转圈圈！）
> 3. **距离数组**：用 `dist[N][N]` 记录从起点到每个位置的步数
> 4. **BFS 过程**：
> - 起点 (1,1) 入队，`vis[1][1] = 1`，`dist[1][1] = 0`
> - 每次取队首，往四个方向走
> - 如果新位置没越界、不是围栏、没走过 入队，标记，距离 +1
> - 直到走到 (n,m) 或者队列空了

完整代码：

```
#include <iostream>
#include <queue>
using namespace std;

const int N = 105;

int n, m;
int a[N][N];      // 地图：0=草地，1=围栏
int vis[N][N];    // 标记是否走过
int dist[N][N];   // 记录从起点到这里的步数

// 方向数组：上、下、左、右
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

struct Point {
    int x, y;
};

int bfs() {
    queue<Point> q;

    // 第一步：起点入队，标记
    Point start;
    start.x = 1;
    start.y = 1;
    q.push(start);
    vis[1][1] = 1;
    dist[1][1] = 0;

    // 第二步：队列不空就循环
    while (!q.empty()) {
        // 第三步：取出队首
        Point cur = q.front();
        q.pop();

        // 如果到了终点，直接返回
        if (cur.x == n && cur.y == m) {
            return dist[n][m];
        }

        // 第四步：扩展四个方向
        for (int i = 0; i < 4; i++) {
            int nx = cur.x + dx[i];
            int ny = cur.y + dy[i];
```

```
            // 判断：不越界、不是围栏、没走过
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m
                && a[nx][ny] == 0 && vis[nx][ny] == 0) {
                // 第五步：入队，标记，记距离
                Point next;
                next.x = nx;
                next.y = ny;
                q.push(next);
                vis[nx][ny] = 1;        // 入队时就标记!
                dist[nx][ny] = dist[cur.x][cur.y] + 1;
            }
        }
    }

    // 队列空了还没到终点，说明走不到
    return -1;
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }

    cout << bfs() << endl;

    return 0;
}
```

代码逐行讲解要点：

代码	讲解话术
`queue<Point> q;`	"创建一个队列，排队用的！排的是花园里的位置（坐标点）"
`q.push(start);`	"小明站到起点，开始排队——入队！"
`vis[1][1] = 1;`	"起点标记为'已走过'，不然以后又走回来就绕圈了"
`while (!q.empty())`	"只要队伍里还有人，就继续探索"
`q.front(); q.pop();`	"排在最前面的人出列，轮到他了"
`for (int i = 0; i < 4; i++)`	"朝四个方向看看，上下左右"
`vis[nx][ny] = 1;`	"**入队时就标记**，不要等出队才标记，否则同一个点会被重复入队！"
`dist[nx][ny] = dist[cur.x][cur.y] + 1;`	"新位置的步数 = 当前位置的步数 + 1，多走了一步嘛"

例题 2：马的遍历（进阶 —— 方向数组变化）

题目描述：

在一个 n×m 的棋盘上，有一匹"马"站在第 sx 行第 sy 列。马走"日"字：每次可以横走 1 步竖走 2 步，或者横走 2 步竖走 1 步（和中国象棋的马一样）。请输出马到达棋盘上每个位置的**最少步数**。如果到不了，输出 `-1`。

输入格式：

一行四个整数 n, m, sx, sy（1 ≤ n, m ≤ 400）。

输出格式：

n 行，每行 m 个整数，表示马到每个位置的最少步数。到不了输出 -1。数字之间用空格隔开。

样例输入：

```
3 3 1 1
```

样例输出：

```
0 3 2
3 -1 1
2 1 -1
```

讲解思路：

> 这道题和例题 1 非常像！区别在哪里呢？
> 例题 1 的小明**上下左右走**，4 个方向。
> 这道题的马走**"日"字**，有 **8 个方向**！
> 所以我们只需要把方向数组从 4 个改成 8 个，其他的 BFS 模板**一模一样**！
> 马的 8 个方向是什么呢？（在黑板上画"日"字的 8 个跳法）

```
    (-2,-1) (-2,+1)
(-1,-2)         (-1,+2)
         [马]
(+1,-2)         (+1,+2)
    (+2,-1) (+2,+1)
```

完整代码：

```
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

const int N = 405;

int n, m, sx, sy;
int dist[N][N];   // 最少步数，-1 表示到不了

// 马走"日"字: 8 个方向
int dx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
int dy[] = {-1, 1, -2, 2, -2, 2, -1, 1};

struct Point {
    int x, y;
};

void bfs() {
    // 初始化所有位置为 -1（表示还没到过）
    memset(dist, -1, sizeof(dist));

    queue<Point> q;

    Point start;
    start.x = sx;
    start.y = sy;
    q.push(start);
    dist[sx][sy] = 0;

    while (!q.empty()) {
        Point cur = q.front();
        q.pop();

        // 扩展 8 个方向
        for (int i = 0; i < 8; i++) {
            int nx = cur.x + dx[i];
            int ny = cur.y + dy[i];

            // 不越界 且 没走过
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m
                && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[cur.x][cur.y] + 1;
                Point next;
                next.x = nx;
                next.y = ny;
```

```
                q.push(next);
            }
        }
    }
}

int main() {
    cin >> n >> m >> sx >> sy;

    bfs();

    // 输出结果
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (j > 1) cout << " ";
            cout << dist[i][j];
        }
        cout << endl;
    }

    return 0;
}
```

讲解要点：

要点	说明
方向数组变了	从 4 个方向变成 8 个方向，但 BFS 框架**完全不变**
用 `dist` 代替 `vis`	技巧：`dist[nx][ny] == -1` 既表示"没走过"，又记录距离，**一个数组两个用途**
`memset(dist, -1, ...)`	用 memset 把 dist 全部初始化为 -1，方便判断"到不了"
输出格式	注意输出空格，最后一个数后面没有多余空格

> 同学们发现了吗？BFS 的**代码框架**几乎没变，只是方向数组不同。以后遇到不同的走法，只要改方向数组就行了！这就是**代码模板**的威力。

附：例题源代码与运行结果

例题 1：Grid BFS — Garden Exploration

```
// Given an n*m grid (0=grass, 1=fence), find the minimum
// steps from (1,1) to (n,m). Output -1 if unreachable.
#include <iostream>
#include <queue>
using namespace std;

const int N = 105;

int n, m;
int a[N][N];      // Map: 0=grass, 1=fence
int vis[N][N];    // Visited marker
int dist[N][N];   // Distance from start

// Direction arrays: up, down, left, right
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

struct Point {
    int x, y;
};

int bfs() {
    queue<Point> q;

    // Step 1: Enqueue the start point and mark it
    Point start;
    start.x = 1;
    start.y = 1;
    q.push(start);
    vis[1][1] = 1;
    dist[1][1] = 0;

    // Step 2: Loop while queue is not empty
    while (!q.empty()) {
        // Step 3: Dequeue front element
        Point cur = q.front();
        q.pop();

        // Check if we reached the destination
        if (cur.x == n && cur.y == m) {
            return dist[n][m];
        }

        // Step 4: Expand in 4 directions
        for (int i = 0; i < 4; i++) {
            int nx = cur.x + dx[i];
```

```
            int ny = cur.y + dy[i];

            // Check: in bounds, not a fence, not visited
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m
                && a[nx][ny] == 0 && vis[nx][ny] == 0) {
                // Step 5: Enqueue, mark, record distance
                Point next;
                next.x = nx;
                next.y = ny;
                q.push(next);
                vis[nx][ny] = 1;                        // Mark on enqueue!
                dist[nx][ny] = dist[cur.x][cur.y] + 1;
            }
        }
    }

    // Queue empty and destination not reached
    return -1;
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }

    cout << bfs() << endl;

    return 0;
}
```

Sample Input

```
5 5
0 0 1 0 0
0 0 0 0 1
1 0 1 0 0
0 0 0 1 0
0 1 0 0 0
```

Expected Output

```
8
```

例题 2：Knight Traversal (8-direction BFS)

```
// On an n*m chessboard, a knight starts at (sx,sy). The knight
// moves in an "L" shape (8 directions). Output the minimum steps
// to reach every cell. Output -1 for unreachable cells.
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

const int N = 405;

int n, m, sx, sy;
int dist[N][N];

// Knight moves in "L" shape: 8 directions
int dx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
int dy[] = {-1, 1, -2, 2, -2, 2, -1, 1};

struct Point {
    int x, y;
};

void bfs() {
    // Initialize all distances to -1 (not reached)
    memset(dist, -1, sizeof(dist));

    queue<Point> q;

    Point start;
    start.x = sx;
    start.y = sy;
    q.push(start);
    dist[sx][sy] = 0;

    while (!q.empty()) {
        Point cur = q.front();
        q.pop();

        // Expand 8 directions
        for (int i = 0; i < 8; i++) {
            int nx = cur.x + dx[i];
            int ny = cur.y + dy[i];

            // In bounds and not visited
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m
                && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[cur.x][cur.y] + 1;
```

```
                Point next;
                next.x = nx;
                next.y = ny;
                q.push(next);
            }
        }
    }
}

int main() {
    cin >> n >> m >> sx >> sy;

    bfs();

    // Output results
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (j > 1) cout << " ";
            cout << dist[i][j];
        }
        cout << endl;
    }

    return 0;
}
```

Sample Input

```
3 3 1 1
```

Expected Output

```
0 3 2
3 -1 1
2 1 -1
```